

2D to 3D Conversion: Vectors for Enhancing Game Development Asset Creation Pipelines

I. Introduction:

One of the most time-consuming and daunting aspects of game development, especially for small studios and solo developers, is asset creation. Building a world that comes alive for a player is no small task, and without large teams of artists, animators, and expensive software, game developers face difficult tradeoff decisions in prioritizing the allocation of their resources (time, money, computational power, etc.). According to game developer posts on social media and forums, depending on the genre and art style, asset creation can take anywhere from 30% to 90% of a game's total development time [1] [2] [3] [4], leading to multiple fields of research being developed in an effort to produce more efficient asset production pipelines [5] [6] [7]. As an aspiring game creator who is often thwarted in my plans by the intimidation of being an artist, animator, designer, engineer, tester, producer, and more, I have become increasingly interested in simplifying as many processes in the game development pipeline as possible, so I can focus on the big picture stuff when designing my games.

My focus in this research is 2D to 3D conversion, which was first inspired by a YouTube short I saw almost a year ago about Adobe Turntable. Watching the conversion from a 2D drawing to 3D rotation in real-time and hearing the excited audience reaction sparked a meaningful thought in my game-oriented mind: what if I could draw a character or object once and generate a 3D game object from it? That would *significantly* cut down the asset creation timeline in making a game. As it turns out, Adobe has made significant progress in the last year since Turntable first came out, and they have released an SVG to 3D conversion pipeline in their Project Neo tool and have tuned up Turntable in the process.

This research project explores both Adobe tools in depth and outlines their capabilities, limitations, and uses within a game development pipeline context. It also addresses the work of another research team related to the 2D-to-3D conversion process, demonstrates an algorithmic approach to 2D-3D conversion in an MVP application, and outlines future work to improve this process.

II. Background Information:

i. Goals of Research

The goals of this research topic are threefold:

1. Understand the Adobe tools well through the lens of game development.
2. Form a solid foundation for a thesis topic in robust 2D-to-3D conversion.
3. Make a proof-of-concept application to demonstrate understanding.

ii. Inputs/Outputs of Adobe Tools

Firstly, before we can develop a solid understanding of the tools being analyzed in this project, we must first understand the inputs and outputs of these systems. In the case of Adobe's Turntable feature, which is available only in 2025+ Beta installations of Adobe Illustrator, the only acceptable input is an SVG file (2D vector graphics), and the output is a List<SVG> data structure that contains different view angles of the input SVG file. This data structure is not exportable itself, but any of the views can be exported as an individual SVG file; "rotation" is achieved with a slider controlling the index of the List<SVG> to present different views. Similarly, Project Neo utilizes imported SVG files to build 3D objects in the Neo editor. Users can output .png, .jpg, .mp4, or .svg formats of their 3D creations. Importantly, this output *does not* include .obj or other 3D object formats – more on this limitation later.

iii. SVG Files

SVG – or **S**calable **V**ector **G**raphics – files are XML-based representations of geometry in a 2D space. They are resolution-independent, meaning they scale with screen size without distorting shape or pixelating. SVGs are defined using elements and attributes [8]. Figures 1 and 2 show a sample of SVG code and the produced image as an example.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
3    <!-- A vertical line 30 units to the right of center -->
4    <line x1="150" y1="20" x2="150" y2="180" stroke="blue" stroke-width="4"/>
5  </svg>
```

Figure 1 – SVG Code Sample 1

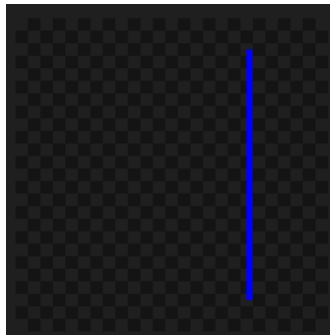


Figure 2 – SVG Image Sample 1

Elements and their attributes are defined within "<>" tags and end with a closing tag – "</element_name>." This example uses only the <line> element, which requires two points – {x1, y1} and {x2, y2} – and can optionally include other attributes like *stroke* and *stroke-width* to further customize the geometry. There are many customization elements and attributes available, but for the purposes of this research, I focused on: <line>, <rect>, <circle>, <polygon>, <path>, and <g> (group). It is important to note that SVG files natively use a top-left-equals-zero coordinate system, so $X_1 = "150"$ and $Y_1 = "20"$ refers to a point that is 150 units right from start and 20 units down from start – visually, this is the top of the line in Figure 2. This distinction becomes important when converting vertex positions to Unity 3D coordinate space. It is also important to note that grouping in SVG occurs two ways: using the *layer* attribute in an element tag or the <g> tag, which is a wrapper that can surround any SVG content.

III. Adobe's Project Turntable:

i. How It Works

Adobe's Turntable feature in Illustrator (Beta 2025+) allows users to select grouped (<g>) vector drawings and generate views using Adobe Firefly – their in-house, art-based generative AI tool. These generated views span 240° of total rotation in 15° increments, which is controlled with a slider that pops up after the tool is done working [9] [10]. Each slider position corresponds to a newly generated SVG file that is completely editable and exportable. Figure 3 shows one of the example inputs used in the Adobe Sneak preview of this feature in 2024 [11], and Figures 4 and 5 show screenshots of some different angles that were generated of this art.



Figure 3 – Example Input 1

Figure 4 – Example Output 1

Figure 5 – Example Output 2

As we can see, it does a really nice job interpreting different views, especially with an art style that lends itself to inexactness like in the example above, but to fully understand the capabilities and limitations of this software, more rigorous tests are needed.

ii. Testing

I used Adobe Firefly's Generate Vectors feature to create some of my own SVG files to test with, from simple geometry to much more complex shapes, and the results were fascinating yet somewhat expected. Simple geometries are nearly impossible for this tool to get correct. Starting with a triangle defined using the <polygon> element, the following figures display the input (Figure 6) and some angled interpretations from the AI (Figures 7 and 8).

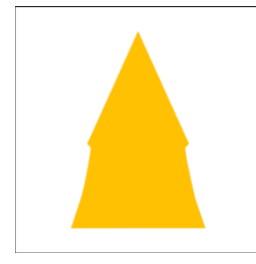
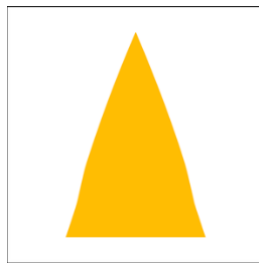
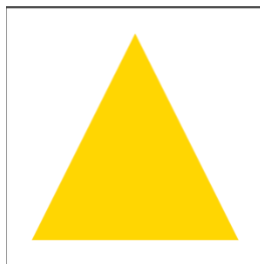


Figure 6 – Example Input 2

Figure 7 – 90° View Output

Figure 8 – 105° View Output

This hallucination is likely caused because the point of this tool is to produce new views by inferring depth information, so when it is met with a geometry that has no depth perspective, it has no idea what to produce. Another simple shape that yielded an interesting hallucination was a circle. In a Reddit post reviewing the functionality of this system, a user made a Mickey Mouse head that mysteriously grew a face [12].

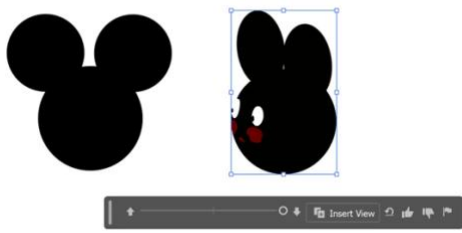


Figure 9 – Reddit User Test 1 Input & Output

Thinking this may potentially be caused because the shape resembles a popular character that likely influenced training data, I decided to test this with my own circle, and the results were the same.



Figure 10 – Example Input 3



Figure 11 – Circle 120° View Output

Interestingly, simple shapes seem to cause hallucinations because there is not enough data for the model to interpret depth and other characteristics of the geometry. It is also worth noting that triangles, rectangles, and other sharp-edged polygons were treated similarly, with some depth being imagined by the AI, while circles alone get faces and are treated as spheres (roughly equal size from all angles). I question how much the training data that was used played a role in this interpretation by the AI – like characters' heads and faces being roughly spherical, but without access to the internal model structure and exact training data, I can only speculate.

In the same Reddit post as mentioned above, the user attempted to import isometric drawings, thinking rigid structure and more detail may lead to a more exact result, but unfortunately the tool did not deliver.



Figure 12 – Reddit User Test 2 Input

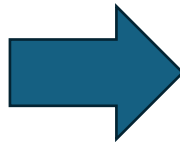


Figure 13 – Reddit User Test 2 Output (2 Angles)

Although the rough shape and linework is there, if we look closely, we can see that much of the clearly defined geometry from the input becomes muddy in the output. Just to name a few things, the “MAN” logo becomes unintelligible blobs, the trailer side strap holes become misshapen, the mirrors change angles between views, and the list goes on. To the finer-eyed game developer or people trying to use this tool for engineering or technical design purposes, this result is unacceptable. However, if the desire is to give the *appearance* of a technical drawing without caring about the specifics, then the result is fine. For example, below are images that mimic a technical style and could be used in the context of a 2D racing game.



Figure 14 – Technical Input 1



Figure 15 – Technical Output 1

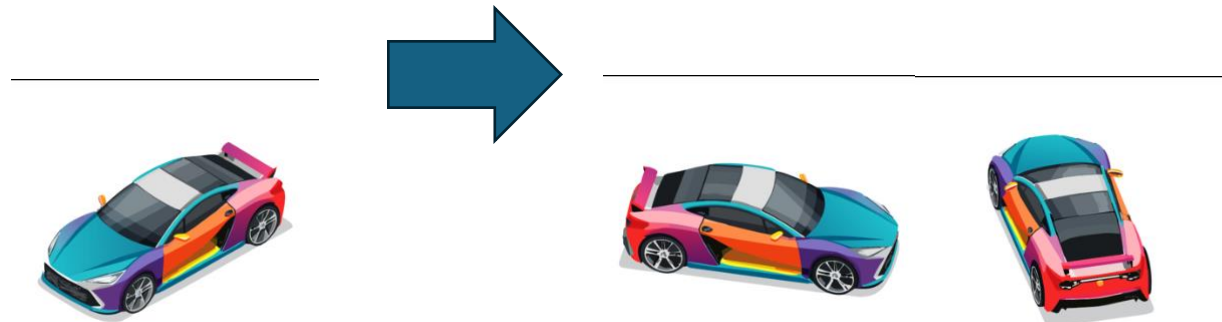


Figure 16 – Technical Input 2



Figure 17 – Technical Output 2

In cases like this, the turntable tool can significantly speed up the art generation process, where an artist may take a matter of hours or days depending on their skill (and cost) to produce the same results, turntable takes a matter of seconds. Not only can it produce different views of imported SVGs, but artists can draw in Illustrator and dynamically generate SVG content as they go, and Adobe Firefly makes text-to-vector generation through prompts a breeze, even allowing parameters like Scene vs. Subject, level of detail, reference assets, and more. Curious about the effect of these parameters on the results, I attempted both higher and lower detail subject creation (see next page).



Figure 18 – High Level-of-Detail Black Dragon: L – Input, M – 120° Clockwise, R – 120° Counterclockwise



Figure 19 – Low Level-of-Detail Black Dragon: L – Input, M – 75° Counterclockwise, R – 120° Clockwise

These two examples used the same prompt (“hyper-realistic black dragon with amber-colored eyes”); the only difference was the level-of-detail parameter (set to min and max values). Both outputs had some inconsistencies like the double tail in the high-detail example or the eye misplacement in the low-detail example, but, overall, I felt a majority of the different views accurately represented the original geometry and with some touch-ups in post, these were very workable and usable vector graphics.

iii. Limitations and Recommended Use in Game Development Pipeline

While I was very impressed with Adobe Turntable’s capabilities in handling complex geometry, it is still quite rough in execution. There appears to be a sweet spot with the level-of-detail in the original vector graphics, and proper grouping of elements plays a huge role in the success of their AI model. With simple shapes, the AI hallucinates often, so a higher level of detail is necessary to allow the AI to distinguish shapes and groups properly, but with that comes increased complexity of the initial design and more work to establish good grouping to guarantee a usable result. In the design of 2D game assets, this tool seems to be very effective when paired with a Unity SVG import tool [13] in a pipeline consisting of the following:

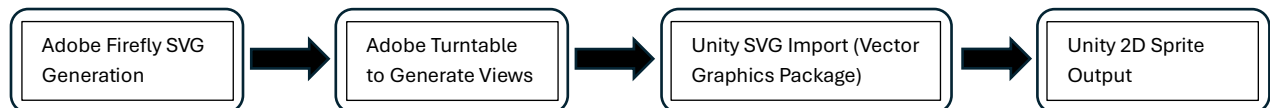


Figure 20 – Suggested Pipeline for 2D Asset Creation Using Adobe Turntable

This enables rapid iteration with prompting to enhance the quality of the input for AI view generation. It is also worth noting that Adobe’s Firefly seems to be far better equipped to produce workable SVG inputs that will operate natively with other Adobe tools, whereas Chat-GPT gave *usable* SVG vector graphics, but quite a bit of tailoring was necessary in grouping the SVG content properly. The car example in Figure 16 was created by Chat-GPT and was tailored by me in Illustrator; the output was surprisingly good considering it was not generated by an Adobe tool, but it did require more manual effort on my part.

One limitation of this tool is the AI generation itself with Firefly. To prevent copyright claims from artists, Adobe trained this model exclusively on publicly available artwork and Adobe-owned IP [14], which leads to a less diverse output range compared to other image generation tools like Chat-GPT, Dall-E, and Google Gemini and limited generation styles in the first step of the pipeline above. There are also many hallucinations produced, especially with flatter geometry, but this can be addressed with a few iterations and better input sanitization. However, the most glaring limitation of this system is the lack of 3D model creation. All inputs and outputs are SVGs, so to create 3D game assets with this tool, a method of 2D to 3D conversion is required. Luckily, Project Neo, released in 2025, works on addressing this issue.

IV. Adobe's Project Neo (Beta)

i. How It Works

Adobe's Project Neo (Beta) extends the capability of their Illustrator software by generating dimensions for artwork in the online Neo editor. Users can import SVG files from Illustrator, directly from a file, or start from scratch with primitive shapes and edit volumetric representations of their 2D geometry, text, or other vector graphics assets [15] [16]. Figure 21 shows the circle example used before in Turntable represented in Project Neo.

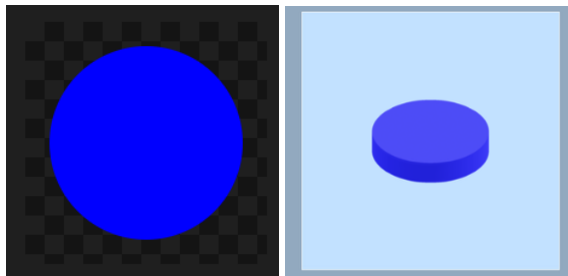


Figure 21 – Circle SVG in Project Neo

Unfortunately, that is about it behavior-wise; it is simple vertical extrusion, and the process is the same for any SVG input. Neo allows users to customize 3D volumes after importing by providing a wide array of gizmos and parameters – including standard TRS manipulation, corner rounding, style manipulators, and lighting effects, among others. After editing the volumetric shapes, users can export in .png, .jpg, .mp4, and .svg, which severely limits the usability of the “3D” shape, so although this is a very promising step in the direction of 2D-to-3D conversion, it is incredibly simple and ineffective for more complex modeling and suffers from severe limitations.

ii. Limitations and Recommended Use in Game Development Pipeline

In its current state, Project Neo is not very useful to anyone other than graphic designers or digital artists mostly because of the output restriction. In the case of game development, .fbx or .obj files would be much more useful, supporting mesh construction at the least and, in the case of .fbx, material properties and animation. Even other applications of 3D modeling, like converting technical vector drawings into 3D CAD models, would require .dwg output. Limiting “3D model” output support to raster graphics or a 2D vector file severely hinders the possible use cases of this software. Furthermore, since every SVG is extruded, there is little room for complexity, and this tool currently seems geared towards “3Difying” artwork to force the *perspective* of volumetric shapes rather than generating any 3D geometry.

Another severe limitation of this software is its usability due to its web-based design. Simple SVGs work no problem, but I found that even small increases in complexity drastically impact the lag of the system while editing. Because a majority of the customization of shapes was laggy, it is difficult to recommend this tool to save time in the creation process. Although, one potential use case in 3D game asset creation using this tool is outlined in the pipeline in Figure 22, but this method is convoluted and

could probably be done faster and less painfully with another method. For reference, Polycam is a 3D modeling software used by over half of Fortune 500 companies that utilizes camera video of an object to render 3D geometry and texture [17] [18].

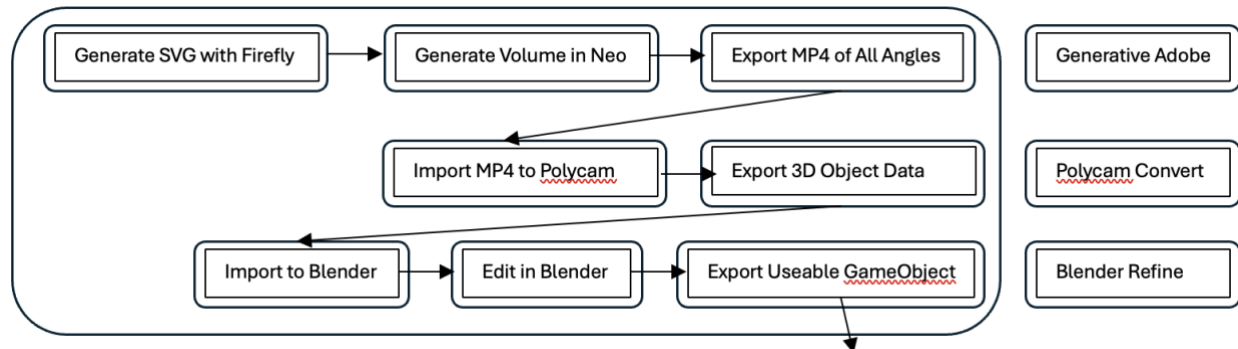


Figure 22 – Suggested Pipeline for 3D Game Asset Creation using Adobe Project Neo

V. Results

After developing a strong understanding of Adobe Turntable and Project Neo, analyzing their strengths and weaknesses, and applying them in the context of a game asset creation pipeline, these tools represent a clear step in the right direction for AI-enhanced asset creation pipelines. They also embody the spirit of the question that started me on this research path: how can one drawing turn into a 3D object? The Turntable feature inspires a sense of magic for users, and even though it is not very usable in a 3D context, I thoroughly enjoyed experimenting with it and watching my custom designs “come to life,” even if not in the way I imagined. Project Neo, although immature, does present a very promising future for 2D-to-3D art conversion, but I sincerely hope that Adobe improves upon the design, particularly the latency issues, by introducing a full-scale desktop app.

Although fun to play with, these tools have some severe limitations when it comes to their applications in game asset creation. Firstly, turntable only allows for 240° of movement per generate views request, meaning any further movement would require exporting one of the generated views and recomputing based on that. This double generation process leads to very lossy structures that often lack the characteristics from the original input. One potential counter to this is starting with two inputs with 180° of separation and generating views based on these two inputs instead of using the lossy input from one source – two handmade drawings are still better than 10-20 in terms of efficiency. Secondly, Project Neo only provides geometry extrusion, which makes everything look like stickers or emblems. More options for how to generate 3D geometry like rotational sweeps or symmetrical mirroring could improve the capability significantly. Also, the online editor is really terrible, and it bottlenecks the service by network traffic rather than computational power. A desktop version is needed to be usable.

VI. Related Research

Ultimately, neither of the Adobe technologies provide out-of-the-box 2D to 3D conversion like I initially hoped. Furthermore, due to Adobe’s research protection, AI model obscurity, and other methods Adobe has used to clamp down on their IP exposure, it is nearly impossible to perfectly dissect these applications to develop a thorough understanding of the backend code and what the model is doing to generate views. Additionally, these technologies were created with large teams and years of model training data, something I simply can’t compete with on my own. However, I did uncover some very interesting related research in my quest to understand this conversion.

“A Visibility Algorithm for Converting 3D Meshes into Editable 2D Graphics” by Eisemann, Paris, and Durand was developed in an effort to accurately depict technical 3D CAD models in brochure or drawing form and, conveniently for me, describes the exact opposite process I am trying to create. This research outlines how to convert a mesh into meaningful 2D layers, respecting visibility ordering and handling self- and mutual-occlusion cases by determining cuts to make in the mesh [19]. This research tells us that there is a lot of information loss when flattening a 3D mesh into 2D space, and by analyzing the processes in this algorithm, we can determine exactly what that lost information is and how to restore it by reversing the process.

Firstly, the process begins by projecting the mesh into 2D space in a viewmap and backtracing each cell to determine a set of faces with identical visibility parameters. Then, these cells are treated as an adjacency-occlusion graph, containing information about faces that are near to each other or blocking the view of another face. Based on these interference or occlusion cycles, cuts are made to separate conflicting areas into different layers. These cuts are highly intelligent and take geodesics into account to form curves in each layer. The output is a 2D Vector representation of the 3D shape. Figure 23 below shows visual aids of their rough process.

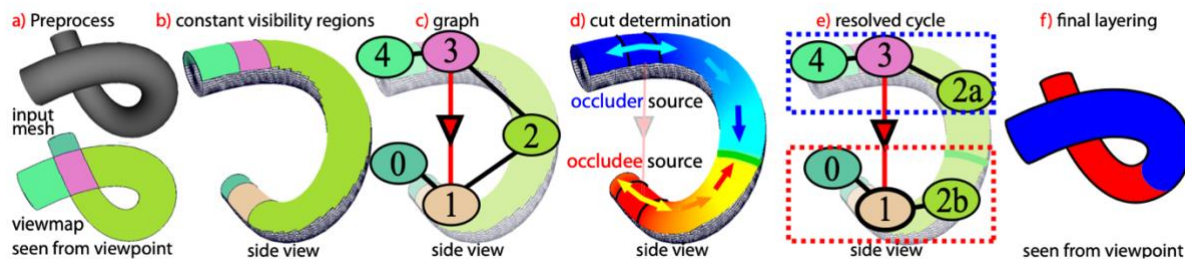


Figure 23 – Visualization of 3D -> 2D Algorithmic Conversion Process

With this information, it is possible to determine an algorithmic approach to what Adobe Turntable and Project Neo provide but output a 3D mesh object instead of useless 2D or raster file formats. Of course, this robust reconstruction of 3D meshes from 2D vectors is an entire thesis topic on its own and involves defining 2D vector regions including stroke semantics instead of constant visibility regions, create an adjacency graph based on layers and groupings in the SVG code, infer visibility and depth ordering from layer placement, grouping, and stroke semantic metadata, and then apply one of numerous 3D geometry techniques to build a model (mesh generation, Poisson reconstruction, point cloud construction). This obviously sounds straightforward but would require many months of research and development to get working. However, this source did inspire me to explore an algorithmic approach to 3D mesh construction based on some user input and assumptions in my proof-of-concept application.

VII. Applying This Knowledge

To model the spirit of these Adobe tools while producing a much more useful result for myself as a game developer, I have created a proof-of-concept application in Unity that successfully inputs SVG files and outputs useable 3D game objects. Here is how the app works (see next page):

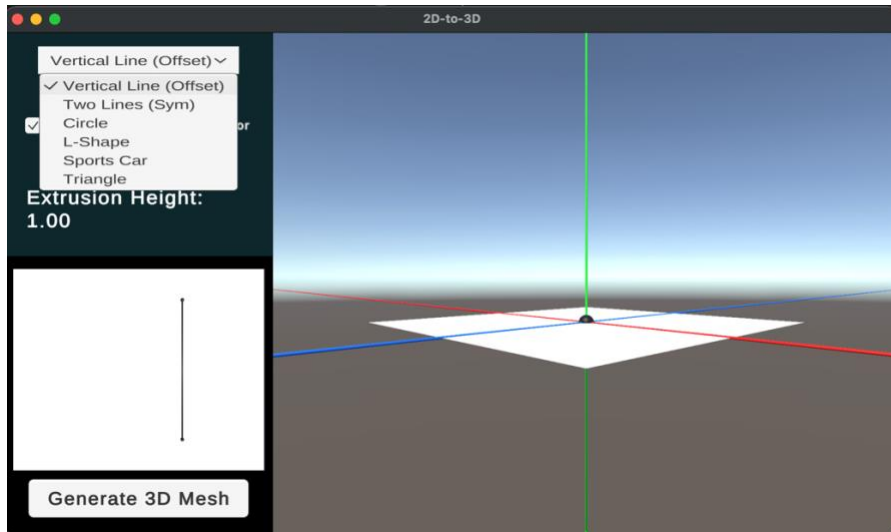


Figure 24 – App Screenshot 1

First, users can select from several different pre-made SVG files to test the application. These SVG files are located in Assets/StreamingAssets/SVG and can be opened in any web browser to see the default 2D artwork. Users can also add any SVG files that are defined in a 200x200 space and properly grouped. Then, users can choose a mesh creation type (extrude – like Project Neo, rotate, or mirror). A preview of the file models a wireframe of the vertex positions taken in by parsing the SVG file. Users may select an extrusion height for extrude mode or a sweep angle value for rotational sweeping. Then, click the generate mesh button.

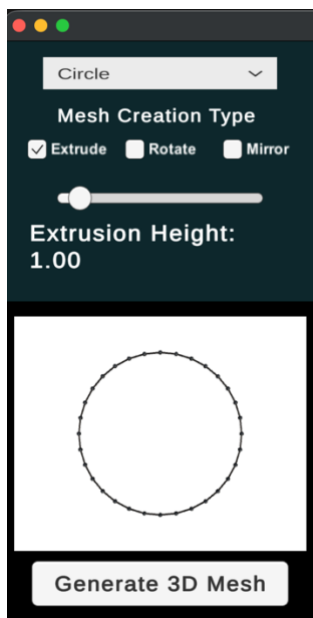


Figure 25 – SVG Control Preview

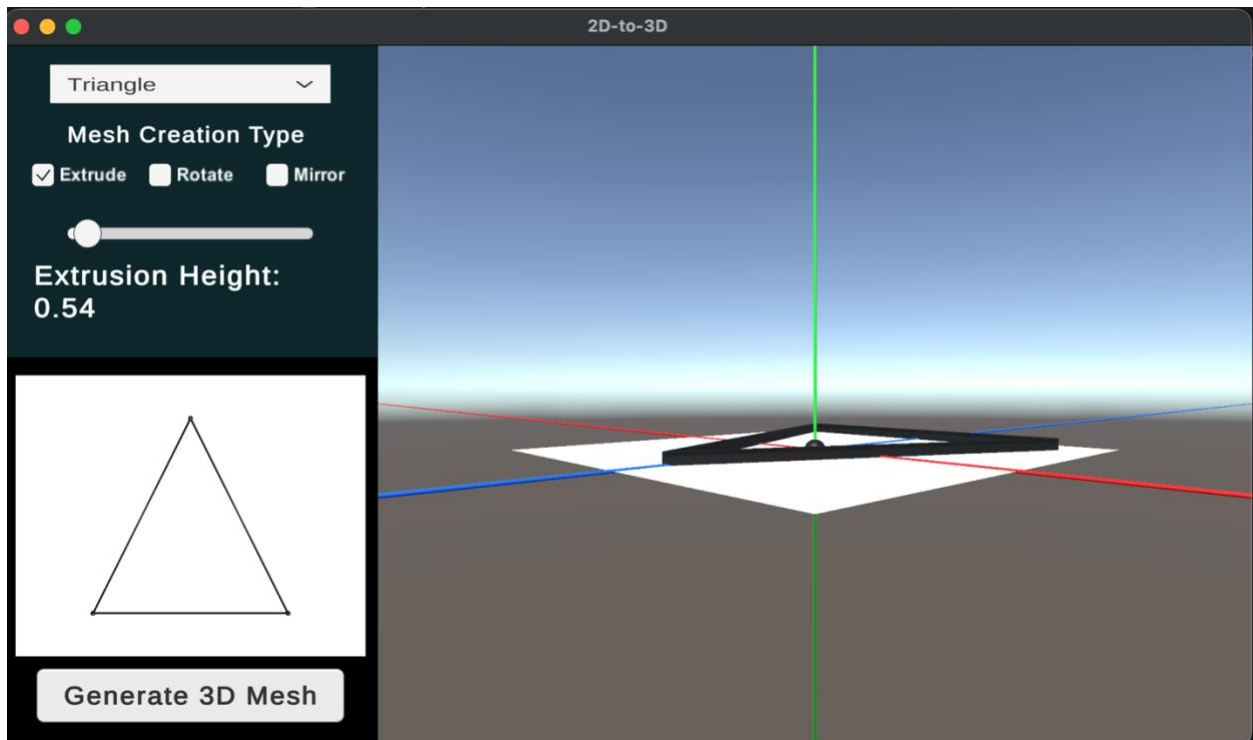


Figure 26 – Example Input and Output 1

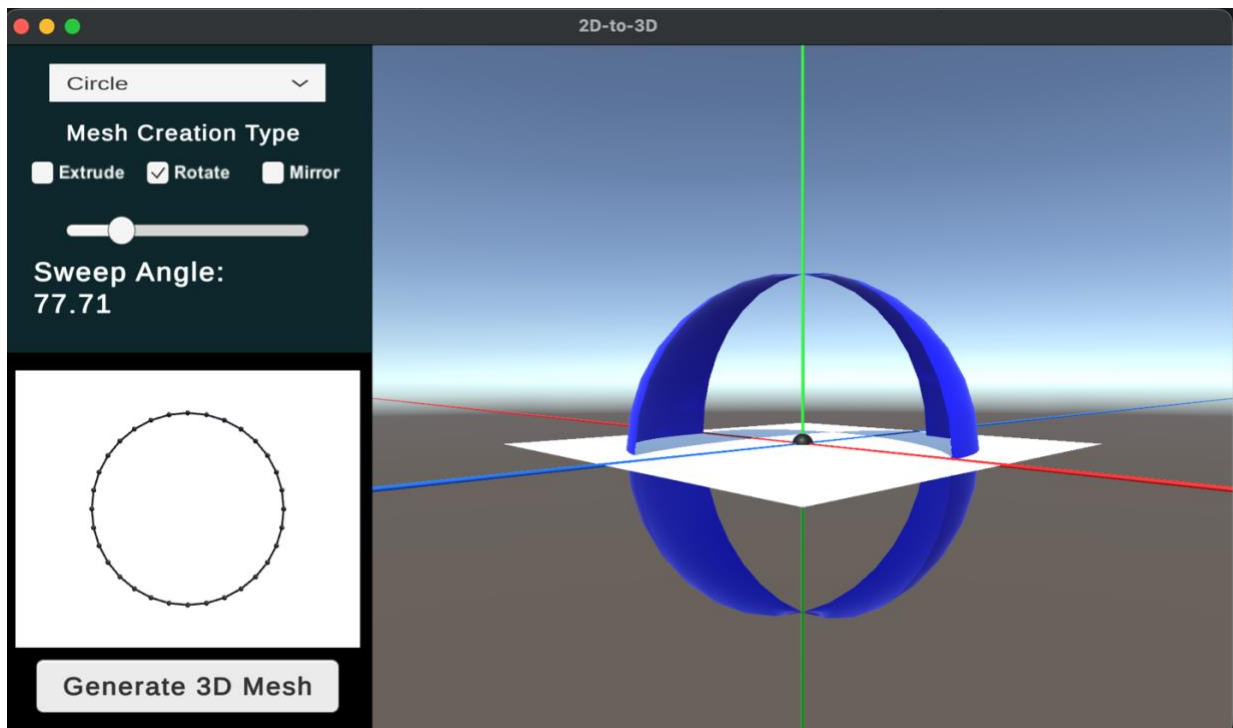


Figure 27 – Example Input and Output 2

On the backend, I am supporting SVG parsing for elements of <polygon>, <line>, <circle>, <rect>, and <path> type, attributes like *stroke*, *fill*, and *color* to determine proper color assignment in Unity, and groupings to determine vertex positions that belong to the same shape. Then, for each shape, I use a triangulation algorithm to build triangle meshes between vertex positions that are supposed to be connected. Finally, material properties that are stored in the metadata are applied at the end.

This mesh, unlike with Adobe Turntable and Neo, is completely usable in a game environment; it is the standard Unity GameObject and can be saved as a prefab after generation. This app is *far* from complete. However, its purpose as a proof-of-concept for algorithmic 2D-to-3D reconstruction gives me hope that one day I will be able to implement a much more robust version that can handle any SVG complexity and give a workable result. It is important to note that in its current state, this app does not support mirror mode yet. It does provide the user with a perspective preview of the SVG at a 45° angle when that toggle is pressed. My general idea for mirroring was to assign many vertices to the geometry (like high-resolution sampling) and allow the preview to be editable, allowing users to reshape the geometry to create depth information. Then, I would treat the preview as one half of the object and mirror it over an axis to form the other side of the geometry. However, due to time constraints and complexity, I did not complete this feature. Moreover, there are several big bugs with this program, namely, it does not currently respect stroke and fill. It treats any color attribute as a stroke color and generates hollow geometries as a result (we can see this in Figure 26 above with the hollow triangle). To make this app more robust, it would need to handle any SVG input possible and decide what to do with it, which is possible to create but was impossible for *me* in this time frame. Overall, I am very satisfied with these results because it proves I understand the core concept of 2D-to-3D reconstruction well enough to dive into more sophisticated methodologies in performing reconstruction.

VIII. Conclusion

This project led me to explore two very interesting Adobe tools – Projects Turntable and Neo (Beta) – in-depth, analyzing their capabilities and limitations as tools for enhancing the efficiency of video game asset creation pipelines. Ultimately, these tools serve as a beacon of hope for the future of AI-enhanced pipelines, but for right now, they underdeliver in comparison with more traditional methods. Some very specific use cases like 2D game asset creation or inefficient 3D pipelines could yield usable results in a game, but it is still not at the stage of readiness where I would recommend changing gears yet, but these tools do offer exciting new prospects in the land of digital art – their true design purpose.

I also analyzed important research related to this concept in the reversed process (3D-to-2D deconstruction), which led to a greater understanding of the information that is lost when moving from 3D to 2D space. Fortunately, by reversing this process, we now understand what information needs to be gathered in order to properly and robustly reconstruct a 3D mesh from 2D vector geometry. Unfortunately, attempting this is extremely difficult and became too complex for the scope of this project.

Finally, I created proof-of-concept application that models very simple 2D (SVG) to 3D (Unity GameObject) conversion, which models the behavior of Project Neo (extrusion) and builds on the functionality by providing a rotational sweep mesh builder as well. While this app is not in a state to be used in an asset creation pipeline yet, it serves as an important cornerstone for an advanced research topic in robust 3D modeling from 2D vector data I am hoping to do for my grad thesis.

References:

- [1] "Where is the time spent?," The Game Development Stack Exchange, Jul. 15, 2010. <https://gamedev.stackexchange.com/questions/508/where-is-the-time-spent> (accessed Dec. 06, 2025).
- [2] "Do indie developers spend most of the time drawing assets for their games or programming them?," Quora, 2019. <https://www.quora.com/Do-indie-developers-spend-most-of-the-time-drawing-assets-for-their-games-or-programming-them> (accessed Dec. 06, 2025).
- [3] "Reddit - The heart of the internet," Reddit.com, 2025. https://www.reddit.com/r/gamedev/comments/4drkyh/what_percentage_of_game_development_teams_are_in/ (accessed Dec. 06, 2025).
- [4] "Reddit - The heart of the internet," Reddit.com, 2022. https://www.reddit.com/r/gamedev/comments/yvzfs2/question_for_indie_game_developers_whats_the/ (accessed Dec. 06, 2025).
- [5] S. Bezzina and Alexiei Dingli, "My Teammate is an AI: Evaluating Generative AI in GameAsset Creation through a Post-GameJam Study," Jun. 2025, doi: <https://doi.org/10.26503/dl.v2025i2.2438>.
- [6] A. Chia, "The artist and the automaton in digital game production," Convergence: The International Journal of Research into New Media Technologies, vol. 28, no. 2, p. 135485652210764, Feb. 2022, doi: <https://doi.org/10.1177/13548565221076434>.
- [7] "Making sure you're not a bot!," Erudit.org, 2025. <https://www.erudit.org/en/journals/loading/2025-v17-n27-loading010083/1118424ar.pdf>
- [8] "SVG Tutorial," www.w3schools.com. https://www.w3schools.com/graphics/svg_intro.asp
- [9] "View 2D objects from new angles," Adobe.com, 2019. <https://helpx.adobe.com/illustrator/desktop/use-generative-ai/view-artwork-from-any-angle.html> (accessed Dec. 01, 2025).
- [10] "Project Turntable," Adobe Research, 2020. <https://research.adobe.com/video/project-turntable/> (accessed Dec. 01, 2025).
- [11] Adobe, "#ProjectTurntable | Adobe MAX Sneaks 2024 | Adobe," YouTube, Oct. 16, 2024. <https://www.youtube.com/watch?v=gfct0aH2COW>
- [12] "Reddit - The heart of the internet," Reddit.com, 2024. https://www.reddit.com/r/AdobeIllustrator/comments/1mtxoy9/turntable_feature_a_quick_overview/ (accessed Dec. 01, 2025).
- [13] "About Vector Graphics | Vector Graphics | 2.0.0-preview.25," Unity 3D Docs, Sep. 24, 2018. <https://docs.unity3d.com/Packages/com.unity.vectorgraphics@2.0/manual/index.html> (accessed Dec. 08, 2025).
- [14] "View of The capabilities of the art-oriented artificial intelligence Adobe Firefly and its visual advantages and disadvantages | Journal 'Fundamental Sciences and Applications,'" Tu-plovdiv.bg, 2024. <https://journals.tu-plovdiv.bg/index.php/journal/article/view/868/54>
- [15] "Project Neo (Beta) overview," Adobe.com, 2019. <https://helpx.adobe.com/project-neo/help/overview.html> (accessed Dec. 01, 2025).
- [16] Adobe Creative Cloud, "Instant Transformations with Project Neo (beta) | Adobe Creative Cloud," YouTube, Oct. 14, 2024. <https://www.youtube.com/watch?v=p6GMIWfuaZA> (accessed Dec. 07, 2025).

[17] "Polycam - LiDAR 3D Scanner," poly.cam. <https://poly.cam>

[18] rileyb3d, "3D Game Assets From a Single Phone Scan in Blender," YouTube, Jun. 29, 2023. <https://www.youtube.com/watch?v=JbKYvLTagn0> (accessed Nov. 12, 2025).

[19] E. Eisemann, S. Paris, and F. Durand, "A visibility algorithm for converting 3D meshes into editable 2D vector graphics," ACM SIGGRAPH 2009 papers, pp. 1–8, Jul. 2009, doi: <https://doi.org/10.1145/1576246.1531389>.